

JAVA DEBUGGING

Q1. DEBUGGING QUESTION

Identify and correct the errors in the following programs. Predict the output after debugging and explain the role of finally.

CODE & OUTPUT

Your Fixed Code

```
1 class Test {
2     public static void main(String[] args) {
3         try {
4             int a = 10 / 0;
5             System.out.println("A = " + a);
6         }
7         catch (ArithmeticException e) {
8             System.out.println("Arithmetic Error");
9         }
10        finally {
11            System.out.println("Finally Block");
12        }
13
14        System.out.println("Program Completed");
15    }
16 }
```

Test Input

stdin input (optional)

Output

```
Arithmetic Error
Finally Block
Program Completed
```

 Pending manual review

Q2. DEBUGGING QUESTION

Debug the following program so that it correctly throws and handles the user-defined exception when the balance is insufficient.

CODE & OUTPUT

Your Fixed Code

```
1 class InsufficientBalance extends Exception {
2
3     InsufficientBalance(String msg) {
4         super(msg);
5     }
6
7
8     static void withdraw(int balance, int amount)
9         throws InsufficientBalance {
10
11         if (amount > balance)
12             throw new InsufficientBalance("Insufficient Balance");
13
14         balance = balance - amount;
15         System.out.println("Remaining Balance: " + balance);
16     }
17
18     public static void main(String[] args) {
19         try {
20             withdraw(5000, 7000);
21         }
22         catch (InsufficientBalance e) {
23             System.out.println(e.getMessage());
24         }
25     }
26 }
```

Test Input

stdin input (optional)

Output

Insufficient Balance

🕒 Pending manual review

Q3. DEBUGGING QUESTION

The following program is intended to safely increment a shared counter using two threads. Debug it so that the final counter value is always 2000.

CODE & OUTPUT

Your Fixed Code

```
1 public class Counter {
2     int count = 0;
3
4     synchronized void increment() {
5         count++;
6     }
7
8     public static void main(String[] args) throws Exception {
9         Counter c = new Counter();
10
11         Thread t1 = new Thread(() -> {
12             for (int i = 0; i < 1000; i++)
13                 c.increment();
14         });
15
16         Thread t2 = new Thread(() -> {
17             for (int i = 0; i < 1000; i++)
18                 c.increment();
19         });
20
21         t1.start();
22         t2.start();
23
24         t1.join();
25         t2.join();
26
27         System.out.println("Final Count = " + c.count);
28     }
29 }
```

Test Input

stdin input (optional)

Output

Final Count = 2000

🕒 Pending manual review

Q4. DEBUGGING QUESTION

The following program attempts to control a thread using deprecated thread-control methods.

CODE & OUTPUT

```
1 class Worker extends Thread {
2     volatile boolean running = true;
3     volatile boolean paused = false;
4
5     public void run() {
6         for (int i = 1; i <= 10 && running; i++) {
7             if (!paused)
8                 System.out.println("Working: " + i);
9
10            try {
11                Thread.sleep(300);
12            } catch (InterruptedException e) {
13                break;
14            }
15        }
16    }
17
18    public static void main(String[] args) throws Exception {
19        Worker w = new Worker();
20
21        w.setPriority(10);
22        w.start();
23
24        Thread.sleep(1000);
25
26        w.paused = true;
27        System.out.println("Thread Suspended");
28
29        Thread.sleep(1000);
30
31        w.paused = false;
32        System.out.println("Thread Resumed");
33
34        Thread.sleep(1000);
35
36        w.running = false;
37        w.interrupt();
38        System.out.println("Thread Stopped");
39    }
40 }
```

stdin input (optional)

Output

```
Working: 1
Working: 2
Working: 3
Working: 4
Thread Suspended
Thread Resumed
```

🕒 Pending manual review